Language Models are Few-Shot Learners

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Dario Amodei

OpenAI

arXiv:2005.14165v1 | NeurIPS 2020

Abstract

Recent work has demonstrated substantial gains on many NLP tasks and benchmarks by pre-training on a large corpus of text followed by fine-tuning on a specific task. While typically task-agnostic in architecture, this method still requires task-specific fine-tuning datasets of thousands or tens of thousands of examples. By contrast, humans can generally perform a new language task from only a few examples or from simple instructions. We show that scaling up language models greatly improves task-agnostic, few-shot performance, sometimes even reaching competitiveness with prior state-of-the-art fine-tuning approaches. Specifically, we train GPT-3, an autoregressive language model with 175 billion parameters, 10 times more than any previous non-sparse language model, and test its performance in the few-shot setting. For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely via text interaction with the model. GPT-3 achieves strong performance on many NLP datasets, including translation, question answering, and cloze tasks, as well as several tasks that require on-the-fly reasoning or domain adaptation. At the same time, we identify some datasets where GPT-3's few-shot learning still struggles, as well as some datasets where GPT-3 faces methodological issues related to training on large web corpora.

Keywords: few-shot learning, language models, scaling laws, in-context learning, GPT-3

1 Introduction

The recent trend in natural language processing has been to pre-train a large model on a broad corpus and then fine-tune it on a downstream task. This recipe has produced large gains on benchmarks such as SuperGLUE, but it requires a labeled dataset for every task and a separate model for every task. In this paper we study a different setting: a single model is given only a natural language description of the task and a small number of demonstrations, and it must perform the task without any parameter updates. We call this in-context learning.

We train GPT-3, a 175 billion parameter autoregressive language model, and evaluate it on more than 40 benchmarks in this few-shot setting. The model has 96 layers, a hidden size of 12288, 96 attention heads, and a context window of 2048 tokens. Training used 10000 NVIDIA V100 GPUs and roughly 300 billion tokens drawn from a mixture of Common Crawl, WebText2, Books1, Books2, and Wikipedia. To our knowledge this is the largest dense language model trained to date.

2 Approach

Our approach combines a scaled-up Transformer language model with a evaluation protocol that requires no fine-tuning. We describe the few-shot setting, the model architecture, and the training data in the following subsections.

2.1 Few-Shot Setting

We evaluate the model in three settings. In zero-shot learning the model is given only a task description and no examples. In one-shot learning it is given one demonstration, and in few-shot learning it is given a small number of demonstrations, typically between 10 and 100. In all cases the demonstrations are concatenated into the prompt, and the model generates the answer autoregressively. No gradient updates are performed, and the same model is used for every task.

This protocol has the advantage that it measures the ability of a single model to adapt to new tasks at inference time, which is much closer to how humans use language. It also removes the need for task-specific architectures and large labeled datasets. The protocol has the disadvantage that the choice of prompt and demonstrations can strongly affect performance, and we therefore report results for several prompt formats where possible.

2.2 Model and Architecture

GPT-3 is an autoregressive Transformer language model with 175 billion parameters, 96 layers, a hidden size of 12288, and 96 attention heads. It uses

alternating dense and locally banded sparse attention layers, and the context window is 2048 tokens. The model is trained with the same objective as GPT-2, namely predicting the next token, and no task-specific heads are added. Table 1 compares the eight model sizes that we train, from 125 million to 175 billion parameters.

We use a byte-level byte pair encoding vocabulary of 50257 tokens. The model is trained with a batch size of 3.2 million tokens, a learning rate that is warmed up over the first 1 percent of training and then decayed, and mixed precision. Training the largest model required 10000 NVIDIA V100 GPUs and roughly 300 billion tokens; the total compute is about 3.14e23 FLOPs. We do not fine-tune the model on any downstream task.

Table 1: The eight GPT-3 model sizes. All models use the same architecture and training data.

Model	Params	Layers	d_model	Heads
GPT-3 Small	125M	12	768	12
GPT-3 Medium	350M	24	1024	16
GPT-3 Large	760M	24	1536	16
GPT-3 XL	1.3B	24	2048	24
GPT-3 2.7B	2.7B	32	2560	32
GPT-3 6.7B	6.7B	32	4096	32
GPT-3 13B	13B	40	5140	40
GPT-3 175B	175B	96	12288	96

2.3 Training Data

The training data is a mixture of five corpora: Common Crawl, which is filtered and deduplicated; WebText2, the extended version of the WebText corpus used to train GPT-2; Books1 and Books2, two book corpora; and Wikipedia. The mixture is sampled with weights that favor higher-quality sources, and the total amount of training data is roughly 300 billion tokens. We remove benchmark test sets from the training data to avoid contamination, and we report the results of a contamination analysis in the appendix.

3 Results

We evaluate GPT-3 on language modeling, question answering, reading comprehension, translation, commonsense reasoning, and arithmetic. In this section we highlight a few representative results; the full set of results is reported in the appendix. Across tasks, performance improves smoothly with model size, and the largest model is often competitive with fine-tuned systems.

3.1 Language Modeling and Cloze Tasks

On LAMBADA, GPT-3 achieves 86.4 percent accuracy in the few-shot setting, compared with 63.24 percent for GPT-2. On the Children's Book Test the model achieves 90.1 percent accuracy, and on HellaSwag it achieves 78.9 percent. These results show that the ability to use long-range context continues to improve with scale, and that in-context learning is particularly effective for cloze-style tasks.

3.2 Question Answering

On TriviaQA, GPT-3 achieves 71.2 percent accuracy in the few-shot setting, which is comparable to the performance of fine-tuned models that use retrieval. On the OpenBookQA and ARC datasets the model also performs well, although it struggles on questions that require multi-step reasoning. Table 2 summarizes the results on these benchmarks.

Table 2: Few-shot results on selected benchmarks. All results use no gradient updates.

Task	Metric	GPT-3 175B
LAMBADA	Accuracy	86.4
TriviaQA	Accuracy	71.2
SuperGLUE	Average	87.8
HellaSwag	Accuracy	78.9
WMT-14 EN-FR	BLEU	32.6

3.3 SuperGLUE and Natural Language Inference

On SuperGLUE, GPT-3 achieves an average score of 87.8 in the few-shot setting, which is close to the performance of fine-tuned models and well above the previous zero-shot state of the art. The largest gains come from tasks that can be formatted as cloze questions, such as BoolQ and MultiRC, while tasks that require careful handling of entailment labels remain difficult.

3.4 Arithmetic and Symbolic Reasoning

GPT-3 can perform two-digit addition and subtraction with high accuracy when given a few worked examples, but its accuracy drops sharply for three-digit arithmetic and for problems that require carrying. On the MATH dataset the model performs poorly, which suggests that in-context learning does not by itself provide the systematicity needed for multi-step mathematics.

3.5 Translation

On WMT-14 English-to-French translation, GPT-3 achieves 32.6 BLEU in the few-shot setting, compared with 11.5 BLEU for GPT-2. On English-to-German the model achieves 24.0 BLEU. These results are below the best supervised systems,

but they show that a language model trained mostly on English can translate with only a handful of demonstrations.

4 Limitations

GPT-3 has several important limitations. First, despite its strong performance on many benchmarks, it still struggles on tasks that require multi-step reasoning, such as arithmetic with large numbers and commonsense reasoning about physical situations. Second, its performance is sensitive to the prompt format, and small changes in wording can produce large changes in accuracy.

Third, the model was trained on a large web corpus, which introduces the risk of memorizing benchmark data and of reproducing biases present in the training text. Fourth, the model is expensive to train and to serve, and its energy consumption is substantial. Finally, like other autoregressive models, GPT-3 can generate fluent but false statements, and it provides no calibrated confidence estimates.

5 Broader Impact

Large language models have the potential to be used for beneficial applications such as writing assistance, translation, and accessibility. They also have the potential to be misused for spam generation, disinformation, and impersonation. Because the model can generate text that is indistinguishable from human writing, it may become difficult to attribute content to its author. We believe that the release of large models should be accompanied by careful analysis of these risks and by mitigations such as output filtering and watermarking.

6 Conclusion

We have shown that a single 175 billion parameter language model can perform a wide range of tasks in the few-shot setting, without any fine-tuning or task-specific architecture. Performance improves smoothly with scale, and on some benchmarks GPT-3 is competitive with fine-tuned systems. At the same time, the model has clear limitations in reasoning, robustness, and efficiency. We believe that scaling up language models is a promising direction, but that it must be combined with better training objectives, retrieval, and evaluation methods to produce systems that are reliable and useful.

$$p(y \mid x) = \prod_t p(y_t \mid x, y_{<t}) \quad (1)$$

$$L = -\sum_t \log p(y_t \mid x, y_{<t}) \quad (2)$$

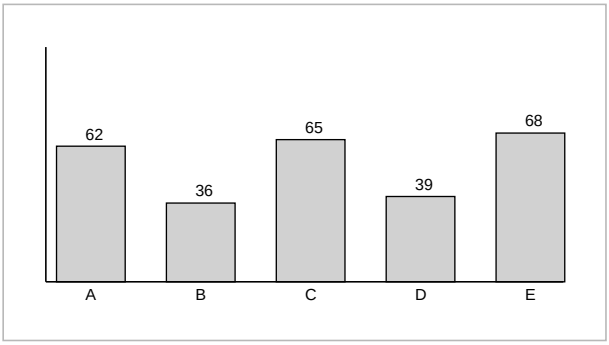


Figure 1: Few-shot performance on selected tasks as a function of model size, showing smooth scaling.

References

[1] Vaswani et al. Attention Is All You Need. NeurIPS, 2017.

[2] Radford et al. Language Models are Unsupervised Multitask Learners. OpenAI Technical Report, 2019.

[3] Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL, 2019.

[4] Radford et al. Improving Language Understanding by Generative Pre-Training. OpenAI Technical Report, 2018.

[5] Kaplan et al. Scaling Laws for Neural Language Models. arXiv preprint, 2020.

[6] Paperno et al. The LAMBADA dataset: Word prediction requiring a broad discourse context. ACL, 2016.

[7] Joshi et al. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. ACL, 2017.

[8] Wang et al. SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems. NeurIPS, 2019.

[9] Zellers et al. HellaSwag: Can a Machine Really Finish Your Sentence? ACL, 2019.

[10] Ott et al. fairseq: A Fast, Extensible Toolkit for Sequence Modeling. NAACL, 2019.

A Appendix: Training Details

All models were trained with the same data mixture and objective. The largest model has 96 layers, a hidden size of 12288, 96 attention heads, and 175 billion parameters, and was trained on 10000 NVIDIA V100 GPUs with a batch size of 3.2 million tokens. The context window is 2048 tokens, and the vocabulary is a byte-level byte pair encoding with 50257 tokens. Training used approximately 300 billion tokens and 3.14e23 FLOPs.

We used Adam with beta_1 = 0.9 and beta_2 = 0.95, gradient clipping at 1.0, and a cosine learning rate schedule with a warmup over the first 1 percent of training. The model was trained in mixed precision, and we used activation checkpointing to fit the largest model in memory. No task-specific fine-tuning was performed for any of the results in this paper.